Msc "Advanced Information systems - Development of Software and Artificial Intelligence, University of Piraeus"

Computational Analysis and Visualization of Twitter Activity During the Mati Wildfire: An Advanced Python Programming Approach

Authors: Konstantinos Mitsionis (mpsp2529), Anastasia Kapellaki

Date: December 22, 2025

Problem Introduction

The mati.csv dataset contains Twitter data related to the tragic Mati Fire, which occurred in the Attica region of Greece in July 2018. This catastrophic wildfire was one of the deadliest in modern European history, causing significant loss of life, destruction of property and widespread environmental damage.Social media platforms, particularly Twitter, played a pivotal role during this event, serving as a medium for information dissemination, public reactions, emergency communications, and the mobilization of relief efforts.

The dataset consists of tweets collected over a specified period of time and includes information about tweet authors, timestamps, geolocation data, textual content, and engagement metrics.t.

In this project, we analyze the Mati dataset in order to understand the dynamics of social media activity during a major crisis. The analysis begins with the filtering of relevant tweets, using text preprocessing and pattern-match techniques to enhance contextual meaning. Subsequently, we examine patterns of activity by analyzing tweet distributions on both a daily and hourly basis, allowing the detection of bursts over time. Also, we analyze user behavior by identifying highly active users and examining interaction metrics, in order to separate user's activity and content impact.

Filtering Irrelevant Tweets

The goal of this section is to develop a computational mechanism for identifying and removing tweets that are unrelated to the natural catastrophe in Mati by analyzing their textual content.

First, in order to ensure the reliability of text-based filtering, we preprocess the textual data of the dataset to reduce noise. At this point we remove unnecessary symbols, emojis, small words, extra spaces etc. As a result, during the preprocessing stage, the textual content of the tweets is cleaned to remove elements that do not contribute to semantic interpretation.

In addition, stop words and non-alphanumeric tokens are removed, while punctuation is retained where appropriate. The objective of this step is to preserve the meaningful linguistic content of each tweet while eliminating elements that could negatively affect text-based filtering.

The results of the two preprocessing steps are illustrated below.

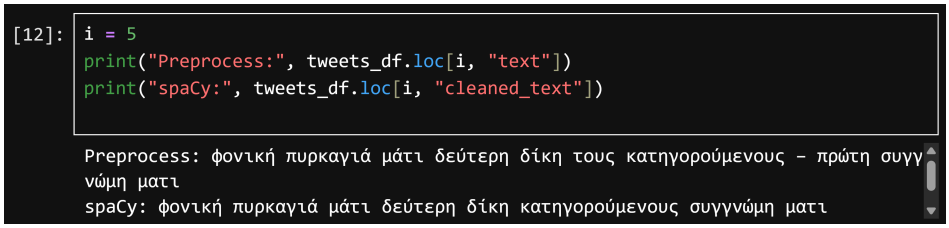


Figure 1: Dataset text after two prerpocessing steps.

Now, text is ready to be fed into a rule-based filtering approach inspired by bag-of-words representations. The filtering logic is designed to ensure that retained tweets explicitly refer to the wildfire event in Mati or in nearby areas.To achieve this, each tweet is required to satisfy two complementary criteria: the presence of a location-related reference associated with Mati or surrounding regions, and the occurrence of at least one keyword related to the

wildfire event. The filtering process was refined iteratively through multiple executions of the algorithm. After each iteration, samples of excluded tweets were manually investigated in order to detect missing patterns or relevant keywords. In those cases, the keyword list was expanded accordingly.

The final text-based filtering algorithm is presented below.

```python
def use_bag_of_words(df):
    place_pat = r"(μάτι|ματι|νέα\s*μάκρη|νεα\s*μακρη|ραφήνα|ραφινα|ανατολικ(ή|η)\s*αττικ(ή|η))"

    event_pat = r"(" + "|".join([
        "φωτιά","φωτια","πυρκαγι","καπν","καμεν","καταστροφ",
        "νεκρ","θυμα","θύμα","αγνοουμ","τραυματ",
        "εκκενωσ","πυροσβεσ","διασωσ","θαλασσ",
        "δικη","ευθυν","κατηγορουμ","κατηγορ","εγκλημα",
        "καταδικ","αθω","εγκλωβ","βοήθεια","βοηθεια","κινδυν"
    ]) + r")"

    df_filtered = df[
        df["cleaned_text"].str.contains(place_pat, regex=True, na=False)
        &
        df["cleaned_text"].str.contains(event_pat, regex=True, na=False)
    ]
    return df_filtered
```

Listing 1: Python implementation of a text-based filtering mechanism for the Mati wildfire

This double-condition strategy significantly reduces the likelihood of retaining tweets that refer to unrelated fires, political discussions, or casual mentions of Mati that are not associated with the disaster. The resulting dataset sizes after the filtering process are illustrated below.

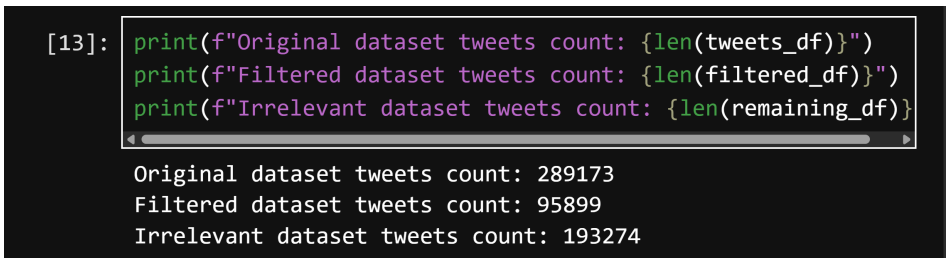
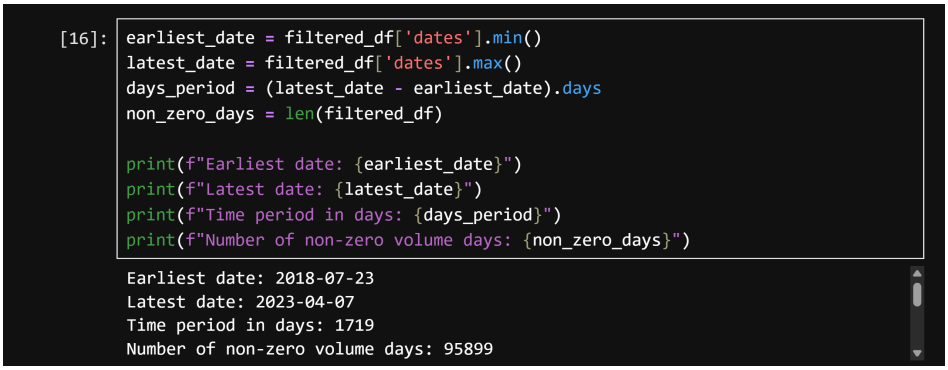


Figure 2: Initial dataset size, filtered dataset size, irrelevant dataset size.

For the remainder of the analysis, only the filtered dataset is used, as it contains tweets that are contextually relevant to the natural catastrophe in Mati and therefore suitable for addressing the subsequent research questions.

Temporal Burst Detection with Moving Averages

The first part of this question deals with creating the daily distribution of tweets over the dataset timeline. Firstly, we analyze the time-span of the filtered dataset, as shown below.

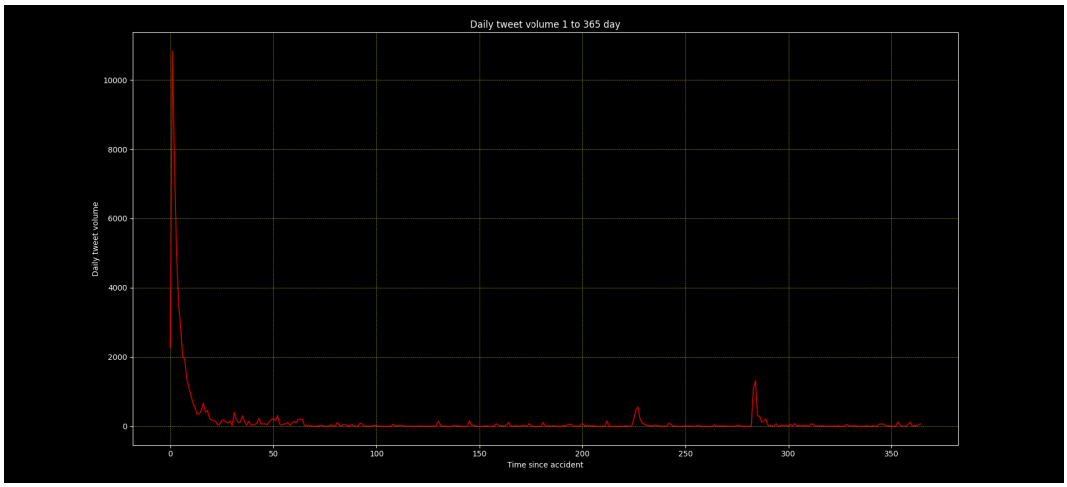


```
[16]: earliest_date = filtered_df['dates'].min()
      latest_date = filtered_df['dates'].max()
      days_period = (latest_date - earliest_date).days
      non_zero_days = len(filtered_df)

      print(f"Earliest date: {earliest_date}")
      print(f"Latest date: {latest_date}")
      print(f"Time period in days: {days_period}")
      print(f"Number of non-zero volume days: {non_zero_days}")

      Earliest date: 2018-07-23
      Latest date: 2023-04-07
      Time period in days: 1719
      Number of non-zero volume days: 95899
```

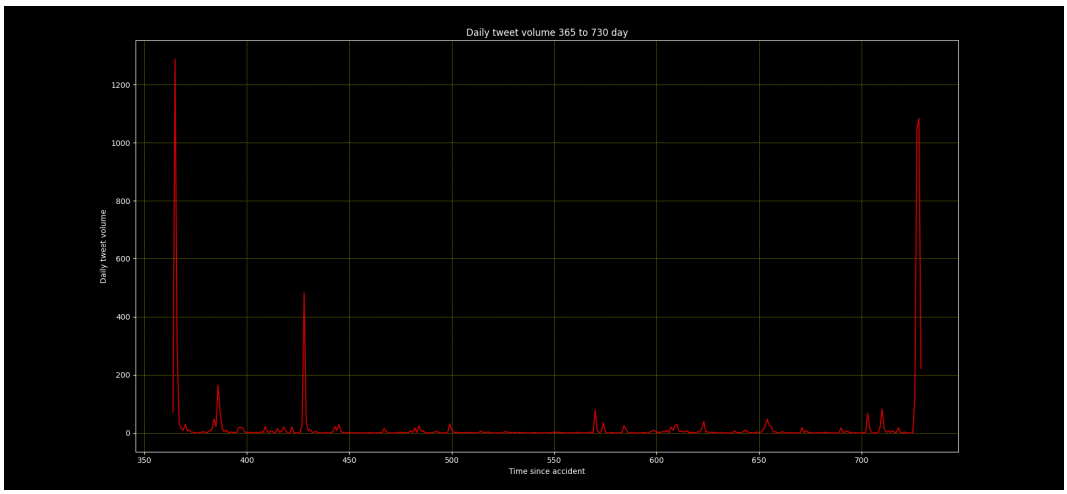
Figure 3: Date ranges for filtered tweets about catastrophe in Mati.

From Figure 13 we observe that the filtered tweets span a period of approximately five years (2018–2023). For a better definition of the distribution plots, we split the date range year by year. The daily volume plots are illustrated in Figures 6 and 7.

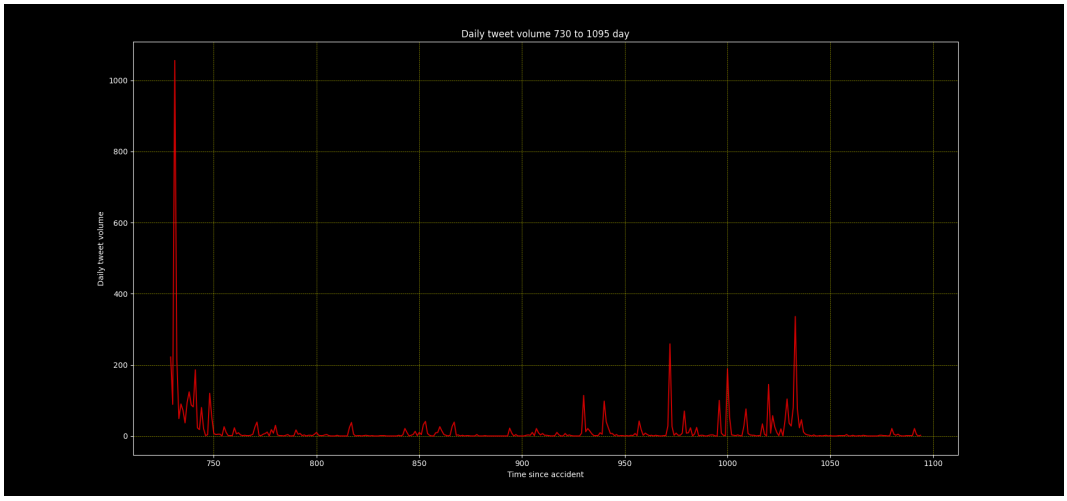
Although the daily volume plots highlight periods of increased Twitter activity, a more systematic methodology is required to detect activity peaks. For this purpose, a rolling average approach is applied to the daily tweet volume, enabling the identification of bursts as deviations from the local activity trend. This method enables a reliable and systematic analysis of Twitter activity related to the Mati catastrophe. In rolling average method we used as $windows\_size = 30$ in order to have a natural timing correlation with the 1-year range of each plot.



(a) First year after the catastrophe

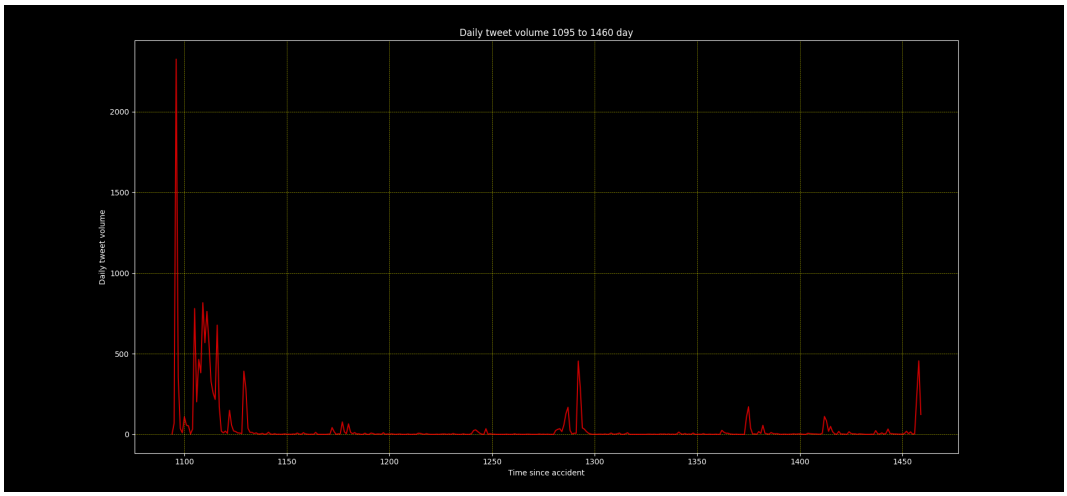


(b) Second year after the catastrophe

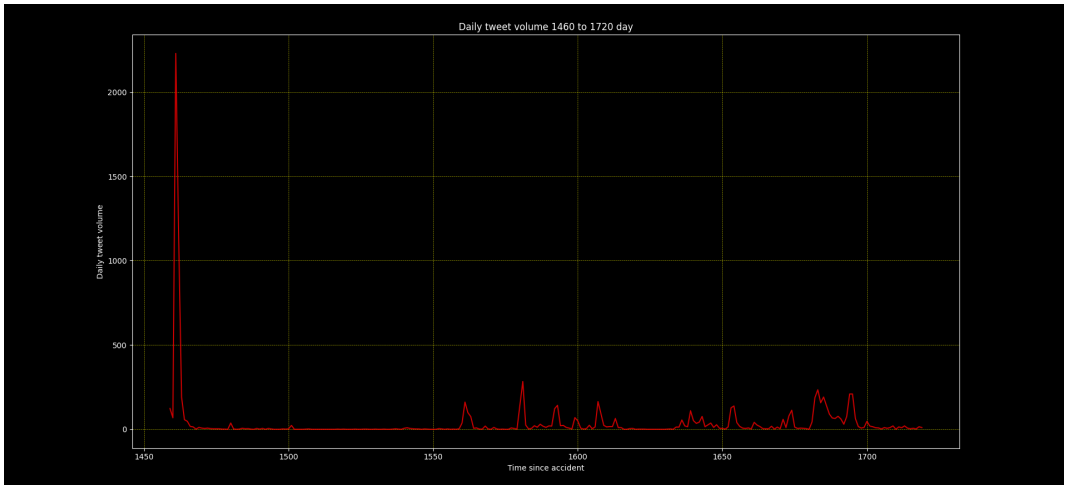


(c) Third year after the catastrophe

Figure 4: Daily volume of filtered tweets related to the Mati wildfire (Years 1–3).



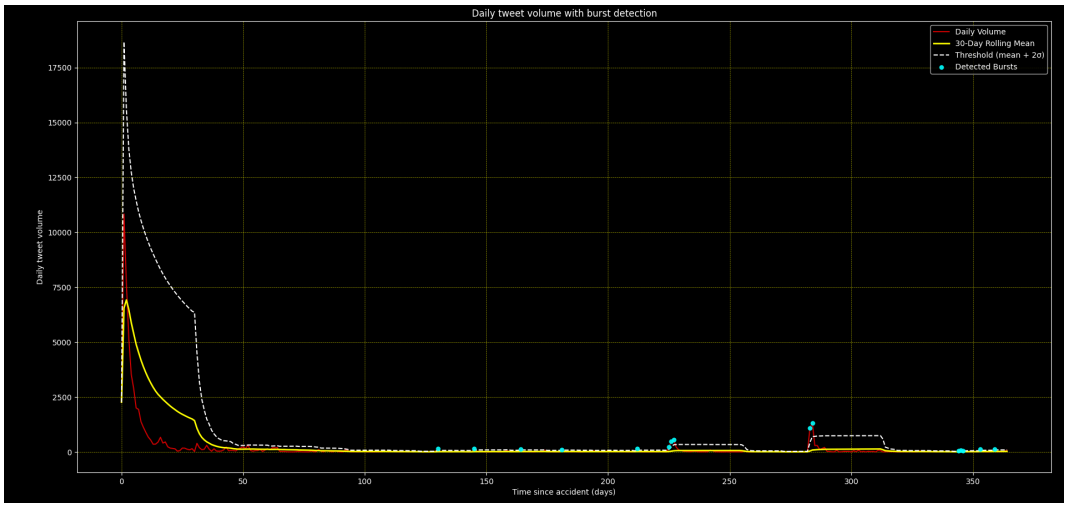
(a) Fourth year after the catastrophe



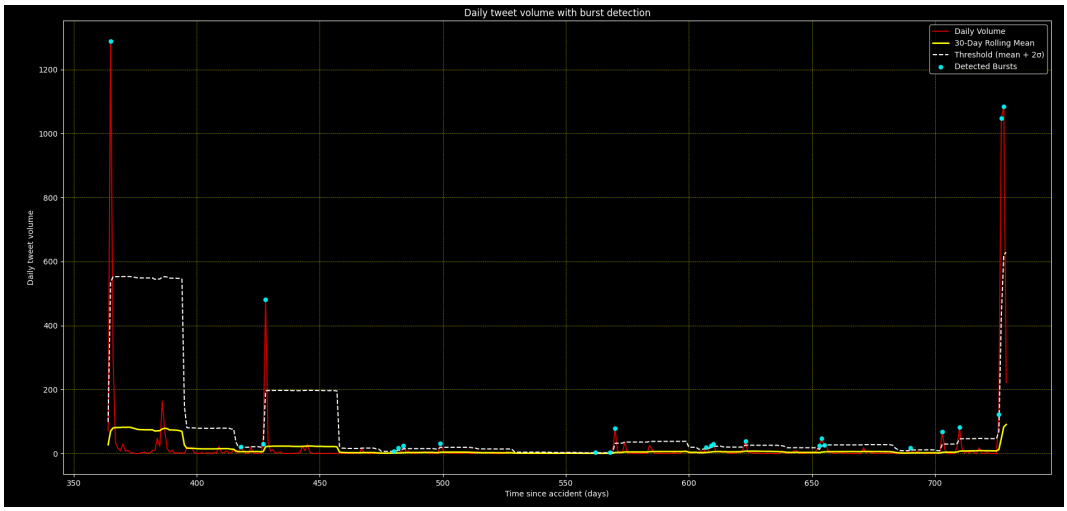
(b) Later years after the catastrophe

Figure 5: Daily volume of filtered tweets related to the Mati wildfire (Years 4–Later).

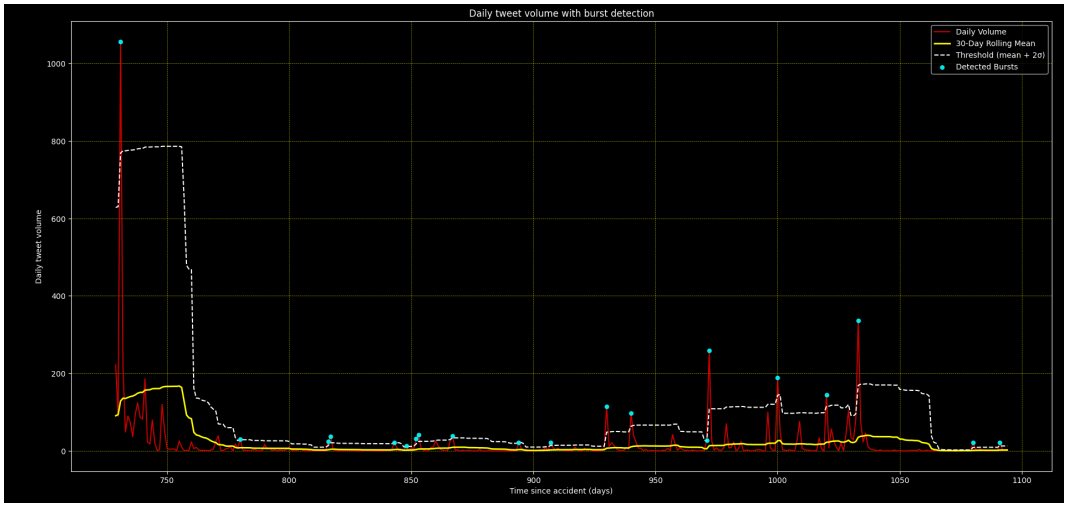
To continue with, we plot volume of data and rolling mean average output, while detecting the bursts after the definition of $threshold = rolling\_mean + 2 * std$. To clarify the existence of bursts, they help us identify statistically abnormal activity relative to its local context. The white dashed threshold in illustrated plots, adapts over time; it is high in periods of sustained attention and low in "quiet" periods.



(a) First year after the catastrophe

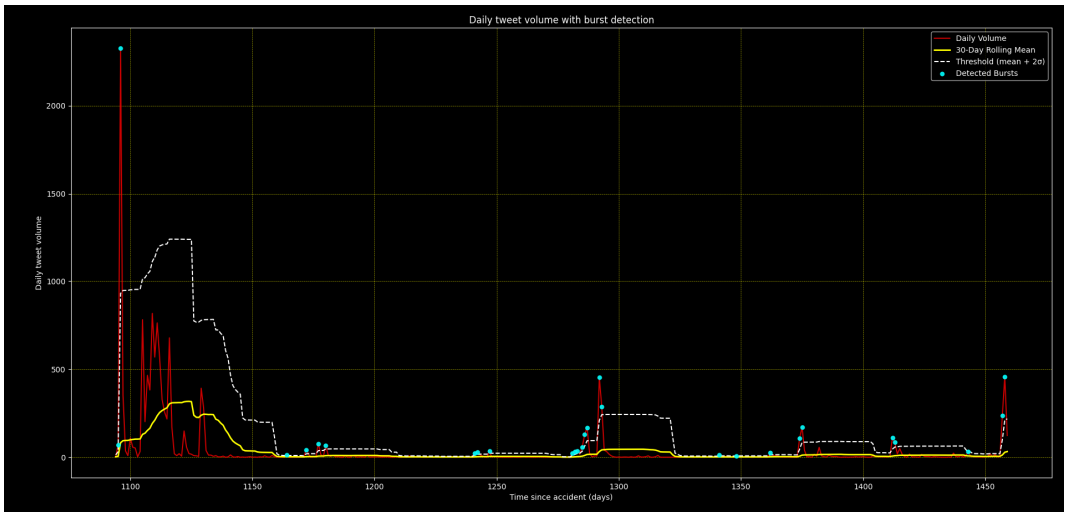


(b) Second year after the catastrophe

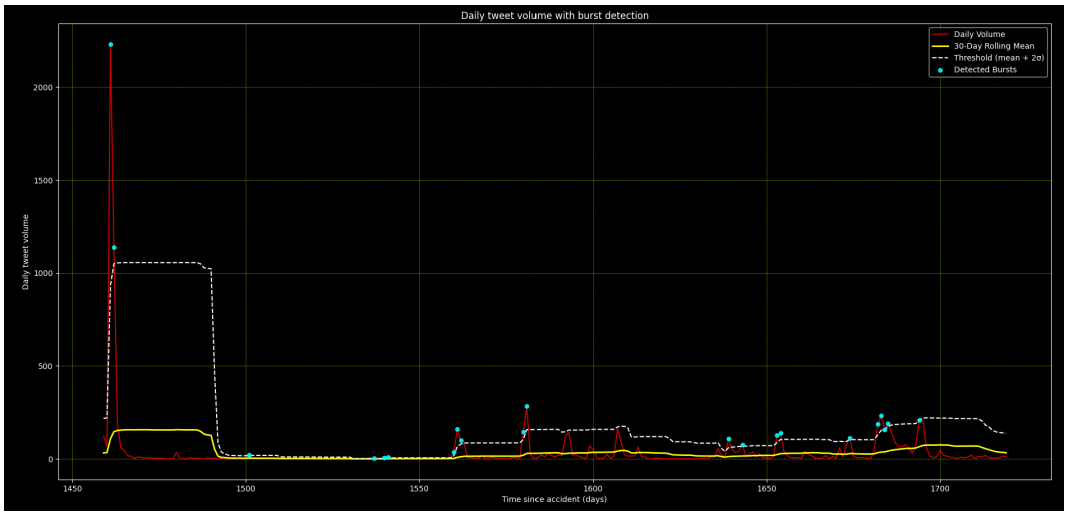


(c) Third year after the catastrophe

Figure 6: Burst detection of filtered tweets related to catastrophe in Mati (Years 1–3).



(a) Later years after the catastrophe



(b) Later years after the catastrophe

Figure 7: Burst detection of filtered tweets related to catastrophe in Mati (Years 4–Later).

It is worth mentioning that burst detection does not provide information about the reasons we identify those spikes. This interpretation is related with external contextual information, such as political developments, judicial proceedings, or media relevants. In addition, the rule-based text filtering strategy, while effective in reducing noise, may exclude relevant tweets that do not explicitly match the predefined keyword patterns.

From the first year after the catastrophe we obtain the biggest part of filtered tweets in our dataset. We detect an obvious spike the first day of the catastrophe. This is an expected output, as users posted tweets right after the beginning of the fire. In addition, we observe several days exceeding the defined threshold during the year, but with much less activity intensity.

We consistently observe activity bursts around the annual remembrance dates of the catastrophe. After the first year, the total tweet volume drops by approximately 90%. At the beginning of the second year, increased activity is observed, possibly associated with the

Greek elections in July 2019.

In the third year (2020), additional information revealed regarding the delayed governmental response, as media released new evidence highlighting responsibilities and omissions.

In the fifth year (2022) one of the possible reasons we detect spikes in activity around catastrophe in Mati is the trial procedure and especially the appeal hearing, which was postponed repeatedly throughout 2022.

The final year included in the analysis was 2023, which was a year with not so many revelations around the catastrophe we detect small bursts with low intensity.

Author Activity Ranking

The first part of this question deals with grouping tweet volume based on author. We group tweets by *author_id* and we calculate *total_tweets*, *time_span* of their activity and *avg_internal_time* between consecutive tweets.

In order to create a ranking weighted system for the most active users, we created some new features from the calculated statistics of each user. Specifically:

```python
def robust_minmax(s):
    lo = s.quantile(0.05)
    hi = s.quantile(0.99)
    #here we check if hi and lo are very close to avoid division by zero. If they are., it means that all values are almost the same, so they dont carry any information.
    if np.isclose(hi, lo):
        return pd.Series(0.0, index=s.index)
    return ((s - lo) / (hi - lo)).clip(0, 1)


def feature_engineering_authors(author_stats):
    df = author_stats.copy()
    df['frequency'] = 1 / (df['avg_interval_hours'] + 1)  # Avoid division by zero
    df['impact'] = df['total_tweets'] / (df['timespan_hours'] + 1)  # Avoid division by zero
    df['frequency'] = df['frequency'].fillna(0.0) #avoid breaking for users with single tweet
    df['impact'] = df['impact'].fillna(0.0) #avoid breaking for users with single tweet
    df['norm_total_tweets'] = robust_minmax(df['total_tweets'])
    df['norm_frequency'] = robust_minmax(df['frequency'])
    df['norm_impact'] = robust_minmax(df['impact'])

    return df
```

Listing 2: Authors feature engineering for weighted ranking system

This normalization rescales values to the range $[0, 1]$ using percentile-based bounds in order to reduce the effect of outliers and ensure numerical stability.

After calculating *total_tweets*, *timespan*, *avg_interval_time* we created some metrics which move in the same direction with our computed statistical metrics. As a result, if *freqeuncy* is high, *avg_interval_hours* is small which leads to an active user. Same for *impact* feature, where big values indicate large numbers of *total_tweets* and small *timespan_hours* from first to last tweet. This lead us to the creation of the weighted ranking system where rank is calculated after we normalize the metrics:

$$\text{activity\_score} = w\_tweet \cdot \text{total\_tweets} + w\_freq \cdot \text{frequency} + w\_imp \cdot \text{impact} \quad (1)$$

For our experiments we selected:

$$w\_\text{tweet} = 0.4, \quad w\_\text{freq} = 0.3, \quad w\_\text{imp} = 0.3$$

Based on our ranking system we evaluate user's activity based on the total volume of their tweets, their posting frequency and the ratio between their total number of tweets posted and the duration of their active period. The distribution of author's volume of tweets is illustrated below.

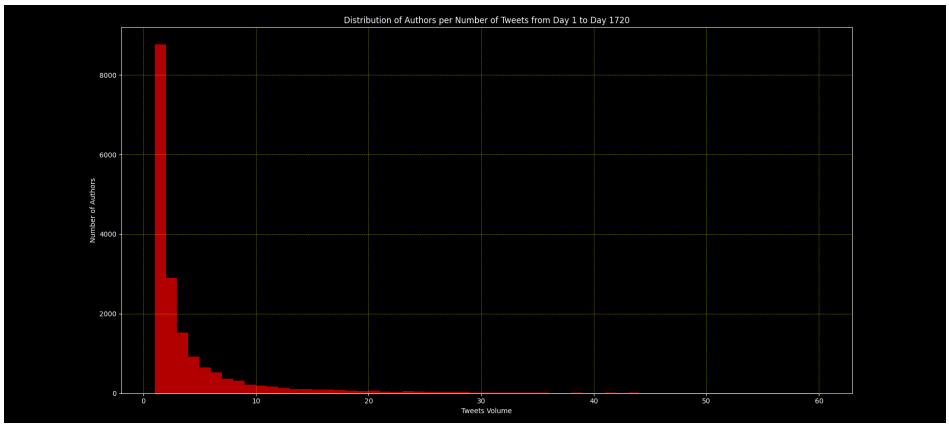


Figure 8: Tweets distribution per author.

Also, we plot the 20 most active users based on our evaluation system, as shown below.

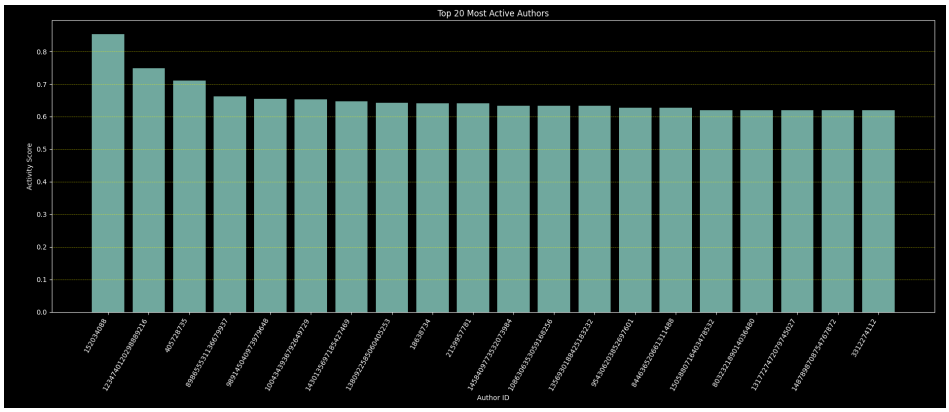


Figure 9: Tweets distribution per author.

Time-of-Day Analysis

This question is related with the tweets volume for each hour of a day. We extract hour for every tweet in our filtered dataset and then we plot the average tweet volume for each hour of the day, as shown below.

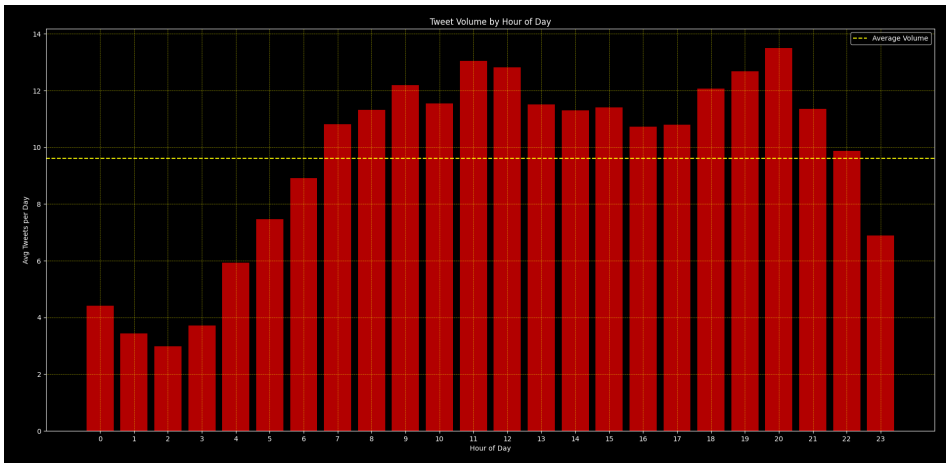


Figure 10: Tweets average volume per each hour of a day.

From the plot above we can extract useful information for user behavior patterns. Tweets volume exceeds average value in the $08:00 - 22:00$ hour range. This is an expected observation, as most people are awake in this hour range. The highest volume identified around 20:00 when probably most people have free time to access Twitter. Volume is quite low, in late night hours.

Engagement Metric Aggregation

In this question, we analyze further authors activity by computing engagement metrics such as likes, retweets and replies. By grouping authors and counting all their engagement metrics, we define a new metric:

$$\text{EngagementRate}_i = \frac{\sum \text{likes}_i + \sum \text{replies}_i + \sum \text{retweets}_i}{\text{total tweets}_i} \tag{2}$$

Based on this metric, we sort users in a descending order. Then, we plot the 20 users with the higher average engagement metrics, as illustrated below.

For a complete analysis and better visualization, we provide tables for top 10 users with the highest overall sum of engagement metrics plus top 10 users with the biggest average engagement metric.

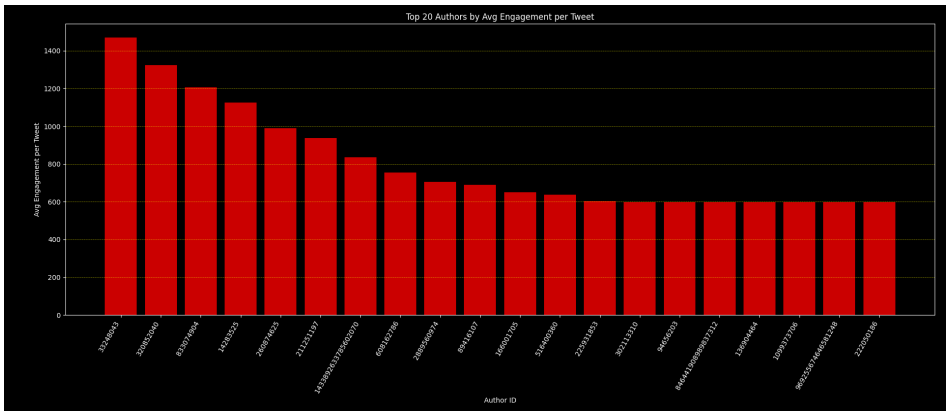


Figure 11: Top 20 users with highest average engagement metrics.

[16]:
```
engagement_df = compute_engagement_metrics(filtered_df)
engagement_df_avg = engagement_df.sort_values(by='avg_engagement_tweet', ascending=False).reset_index(drop=True)
engagement_df_avg.head(10)
```

[16]:

	author_id	total_likes	total_replies	total_retweets	total_tweets	avg_engagement_tweet	total_engagement
0	33248043	1189	34	246	1	1469.0	1469
1	320852040	248	17	1059	1	1324.0	1324
2	833074904	1751	155	505	2	1205.5	2411
3	14283525	867	40	218	1	1125.0	1125
4	260874625	732	87	171	1	990.0	990
5	211251197	827	6	104	1	937.0	937
6	1433892633785602070	681	53	101	1	835.0	835
7	608162786	240	6	510	1	756.0	756
8	2889560974	1126	94	189	2	704.5	1409
9	89416107	1864	110	780	4	688.5	2754

Figure 12: Top 10 users in average engagement metric.

[18]:
```
engagement_df_overall = engagement_df.sort_values(by='total_engagement', ascending=False).reset_index(drop=True)
engagement_df_overall.head(10)
```

[18]:

	author_id	total_likes	total_replies	total_retweets	total_tweets	avg_engagement_tweet	total_engagement
0	3173348733	5605	228	18334	437	55.302059	24167
1	1159260668	9803	605	13283	232	102.116379	23691
2	1298248224	5645	127	15195	268	78.235075	20967
3	2349489651	0	0	15735	346	45.476879	15735
4	1244628291404824578	7173	319	6214	188	72.904255	13706
5	39467121	9612	298	3014	32	403.875000	12924
6	454229130	0	0	12751	124	102.830645	12751
7	3300398932	0	0	12604	244	51.655738	12604
8	486783894	1958	161	10236	232	53.254310	12355
9	950647208017350656	5656	198	6360	139	87.870504	12214

Figure 13: Top 10 users with highest sum of engagement metrics.